PROJECT TECHNICAL REPORT

Smart Queue Manager

An Intelligent Virtual Queue & Priority Counter Management System with ZXing QR Check-In, Real-Time TV Display Board, Dynamic Wait-Time Estimation, Rule-Based No-Show Risk Engine, and Customer Feedback Ratings.

Project Version:	1.0.0 (Production / Academic Presentation)	**Backend Engine:**	Spring Boot 3.3.4 · Java 21 (LTS)
Database:	H2 In-Memory RDBMS (Zero-Config)	**QR Generation:**	ZXing (Zebra Crossing) 3.5.3
Automated Tests:	JUnit 5 · 20/20 Passing (100% Pass Rate)	**Client Architecture:**	React 19 SPA · Tailwind CSS · TypeScript
Author / Presenter:	Software Engineering Engineering Team	**Date of Publication:**	November 2026

Executive Summary

Physical waiting queues in public service environments—including university canteens, outpatient medical clinics, registrar verification counters, and cashier counters—suffer from four persistent operational bottlenecks: (1) lobby crowding and congestion, (2) arbitrary or unfair ordering without consideration for vulnerable or emergency individuals, (3) unpredictable waiting times leading to premature abandonment, and (4) idle counter downtime caused by absent no-shows. **Smart Queue Manager** addresses these challenges by implementing a complete virtual ticketing pipeline with deterministic priority scheduling, adaptive rolling-average wait-time calculation, ZXing-powered mobile QR check-in, privacy-guaranteed public display boards, and customer satisfaction metrics.

1. System Architecture & Component Design

The application strictly adheres to the clean enterprise layered architecture pattern. The system is split into cleanly decoupled layers: REST API Controllers, Data Transfer Objects (DTOs), Business Logic Services, Spring Data JPA Repositories, Domain Entities, and an in-memory database store.

Layer	Component Name	Responsibilities & Behavioral Contract
Controller	QueueRestController CheckInRestController DisplayRestController FeedbackRestController AdminRestController	Exposes RESTful endpoints, handles HTTP request validation (Jakarta Validation), manages serialization envelopes via ApiResponse<T>, and translates domain exceptions to standard HTTP status codes.
Service	QueueService CheckInService DisplayBoardService FeedbackService QrCodeService CustomerService CounterService	Encapsulates pure business logic: priority queue sorting (Comparator), dynamic wait-time calculation, ZXing PNG byte generation, physical lobby check-in validation, privacy-safe display projection, and rule-based no-show risk assessment.
Repository	QueueEntryRepository CustomerRepository FeedbackRepository ServiceCounterRepository QueueHistoryRepository	Spring Data JPA repositories providing transaction-managed CRUD operations, derived query methods, and JPQL aggregations (e.g., average rating calculation, counts by status).
Domain	QueueEntry, Customer, Feedback, ServiceCounter, QueueHistory	JPA entities mapped to relational database tables with foreign-key constraints, enum converters, validation annotations, and text column definitions for QR payloads.
Database	H2 In-Memory Database	Embedded, zero-installation SQL engine running on jdbc:h2:mem:smartqueuedb with web inspection console enabled at /h2-console.

2. Priority Queue Algorithm & Mathematical Model

Standard FIFO queues fail in healthcare or urgent desk contexts where critical conditions demand immediate triage. Smart Queue Manager introduces a 4-tier deterministic priority precedence model:

Priority Level	Value	Precedence Order	Operational Context & Criteria
Emergency	**1**	Highest (Served First)	Severe acute distress, critical medication desks, or immediate supervisor escalations.
VIP	**2**	Served after Emergency	University faculty, visiting guest scholars, or priority cardholders.
Elderly / Senior	**3**	Served after VIP	Senior citizens, pregnant patrons, and mobility-assisted individuals.
Normal	**4**	Standard Virtual Line	Standard FIFO line for regular transactions and canteen orders.

Tie-Breaking & Sorting Implementation

When two customers have identical priority tiers, the queue resolves precedence using the original arrival timestamp (joinedAt). Each service category (Food Order, Doctor Consultation, Document Verification, Payment Counter) maintains an independent queue line so that delays at one counter never interfere with another.

```
Java Comparator Implementation:
Comparator.comparing(QueueEntry::getPriority,
Comparator.comparingInt(PriorityCategory::getValue)).thenComparing(QueueEntry::getJoinedAt);
```

Adaptive Waiting-Time Estimation Formula

Instead of static guess-estimates, expected waiting time is computed dynamically using actual historical throughput:

$$\text{Estimated Wait (minutes)} = \text{People Ahead} \times \text{Average Service Time (minutes)}$$

• **Default Baseline:** 3.0 minutes.
• **Adaptive Rolling Average:** Whenever counter staff marks a ticket as SERVED, the exact service duration (serviceCompletedAt - serviceStartedAt) is recorded into QueueHistory. The rolling average updates as: newAverage = (previousAverage + actualDuration) / 2.
• **Real-Time Re-computation:** All remaining waiting tickets in that department automatically update their queue position and estimated wait time.

3. Enhanced Production Features

Three practical, enterprise-grade capabilities were engineered into the system without compromising existing architecture:

Feature 1: Mobile QR Token Check-In & Arrival Verification

When a patron joins a virtual queue, the system generates a unique QR code encoded with standard JSON attributes:

```
{ "tokenNumber": "D-101", "customerName": "Savita Devi", "serviceType": "DOCTOR_CONSULTATION",
"joinedAt": "2026-11-12T10:30:00" }
```

• **ZXing QR Library Integration:** Generates high-density PNG byte streams without third-party web services.
• **Physical Check-In Desk (/admin/check-in):** Staff can enter token numbers manually or scan customer phone screens.
• **Status Transition to CHECKED_IN:** Distinguishes between customers who joined remotely vs. those physically in the lobby.
• **Turn-Is-Near Alert:** If an unverified customer reaches position 1 or 2, an admin warning triggers: *'Customer has not checked in yet.'*
• **Priority Invariance:** Physical check-in does not alter priority ordering; it simply flags physical presence.

Feature 2: Public Waiting Area Display Board (/display)

Designed specifically for overhead televisions and lobby monitors:

• **Distance-Readable Typography:** Large high-contrast token numbers legible from 25+ feet away.
• **Strict Privacy Guarantee:** Customer names, phone numbers, and past visit frequencies are completely excluded from the public display.
• **5-to-10 Second Auto-Polling:** Keeps all displays synchronized via silent background polling.
• **Audio Notification:** Web Audio synthetic dual-tone chime notification (E5 → C5) when a token is called.
• **Animated Guidance Banner:** *'Please proceed to the assigned counter.'*
• **Fallback State:** Displays *'No customer is currently being served'* when desks are idle.

Feature 3: Customer Feedback & Rating System (/feedback/{tokenNumber})

Allows patrons to evaluate service quality after completing their visit:

• **1 to 5 Star Rating:** Validated integer range with visual stars.
• **Optional Suggestions:** Up to 500 characters with live character-count validation.
• **Strict Precondition:** Feedback can only be submitted for completed (SERVED) tickets.
• **Duplicate Guard:** Enforces a strict one-review-per-token policy.
• **Administrative Analytics:** Computes overall average satisfaction rating and distribution breakdown (5★ down to 1★).

Feature 4: Rule-Based No-Show Risk Heuristic

Instead of black-box ML models, the system uses clear, explainable business heuristics to alert staff:
• **HIGH Risk:** 2 or more historical no-shows (totalNoShows >= 2). Displays red warning banner.
• **MEDIUM Risk:** Exactly 1 historical no-show (totalNoShows == 1).
• **LOW Risk:** 0 historical no-shows.
• *Human-in-the-Loop Principle:* High-risk customers are never dropped automatically; counter staff retains full discretion.

4. Comprehensive REST API Reference

All API endpoints follow RESTful standards and return standardized JSON envelopes containing a boolean success flag, message, payload data, and ISO-8601 timestamp.

HTTP Method & Endpoint	Category	Description & Status Code
POST /api/queue/join	Kiosk	Registers customer and issues alphanumeric token with QR code. (201 Created)
GET /api/queue/{tokenNumber}/qr	Check-In	Returns ZXing-generated PNG image byte stream for token QR code. (200 OK)
POST /api/queue/{tokenNumber}/check-in	Check-In	Validates ticket and marks physical arrival at counter lobby. (200 OK)
GET /api/admin/check-in-status	Check-In	Lists all active tickets with arrival timestamps and check-in status. (200 OK)
GET /api/display/current	Display	Returns currently serving tokens and assigned counter desks. (200 OK)
GET /api/display/service/{serviceType}	Display	Returns privacy-safe display payload for specific department. (200 OK)
POST /api/feedback	Feedback	Submits 1–5 star rating and comment for SERVED customer. (201 Created)
GET /api/admin/feedback/statistics	Feedback	Returns average satisfaction rating and distribution map. (200 OK)
POST /api/queue/{tokenNumber}/serve	Admin	Calls customer to assigned counter and records service started time. (200 OK)
POST /api/queue/{tokenNumber}/complete	Admin	Marks ticket SERVED and updates department rolling average duration. (200 OK)
POST /api/queue/{tokenNumber}/no-show	Admin	Records absentee, increments customer no-show history. (200 OK)
POST /api/admin/reset-day	Admin	Safely archives active tickets as CANCELLED and resets sequence counters. (200 OK)

5. Quality Assurance & Automated Test Results

The project includes a robust JUnit 5 and Spring Boot Starter Test suite. All 20 automated tests pass with a 100% success rate:

Test Suite Class	Tests	Key Test Scenarios Verified	Result
QrCodeServiceTest	2	Validates ZXing PNG byte stream generation and JSON payload structure.	**PASSED**
CheckInServiceTest	3	Validates check-in by token number, duplicate check-in prevention, and terminal status rejection.	**PASSED**
DisplayBoardServiceTest	1	Validates public TV projection payload and strict privacy guarantees (zero names/phones).	**PASSED**
FeedbackServiceTest	5	Validates 1–5 star boundaries, 500-char comment constraints, SERVED status requirement, and duplicate review guards.	**PASSED**

CustomerServiceTest	4	Validates rule-based no-show risk transitions: LOW (0), MEDIUM (1), and HIGH (>=2).	**PASSED**
QueueServiceTest	3	Validates priority ordering, secondary timestamp tie-breaking, dynamic wait formulas, and duplicate ticket guards.	**PASSED**
StatisticsServiceTest	1	Validates daily throughput counts, busiest department aggregation, and safe reset day archiving.	**PASSED**
SmartQueueIntegrationTest	1	End-to-end integration test validating priority jumps, servicing, dynamic wait updates, and no-show risk escalation.	**PASSED**
Total Automated Tests	**20**	**Full regression suite covering all domain logic and edge cases**	**BUILD SUCCESS**

6. Local Setup & Execution Guide

The system can be executed locally with a single terminal command using Maven and Java 21:

```
# 1. Run Spring Boot Backend (Port 8080 with H2 DB) mvn spring-boot:run # 2. Run Test Suite mvn test # 3. Access In-Memory H2 Database Console URL: http://localhost:8080/h2-console | JDBC: jdbc:h2:mem:smartqueuedb | User: sa | Password: (blank) # 4. Demo Admin Credentials Password: admin123 (Notice: In production, configure Spring Security OAuth2 / JWT)
```

7. Conclusion & Presentation Readiness

Smart Queue Manager represents a complete, robust, and extensible enterprise solution for high-throughput service environments. By uniting prioritized queuing logic, ZXing mobile QR code check-in, privacy-guaranteed public display boards, adaptive service duration learning, rule-based no-show heuristics, and post-service customer feedback in a unified Spring Boot and React stack, the project provides an exceptional foundation for both academic capstone defense and practical commercial deployment.